

ADR 0060 — `refuse_if_git_busy`

verifies liveness before trusting `index.lock`

Date: 2026-08-18 Status: Accepted — implemented

Extends `coordinator.rs`'s own doc comments and ADR 0019 (detection, not exclusion, of outside git). Complements ADR 0058, which measured that `index.lock` is *not* held during hook execution for `CommitOnHead` / `AmendCommit` specifically — this decision covers every other way the lock can be orphaned, for every caller of `refuse_if_git_busy`, not only those two operations.

Context

An independent adversarial refutation of the #72 work (2026-08-18) confirmed a real defect: `refuse_if_git_busy` tested only whether `<git-dir>/index.lock` *exists*, then told the browser-only user "Another git process is working in this repository — wait for it to finish and try again" — a fact the check could not actually know. A lock orphaned by a process that already died (a killed hook, an OOM-kill, a crash mid-write) is indistinguishable from a live write under an existence-only test, and once wrong the message can never become right again: every following request against that repository is refused, forever, recoverable only by a human with shell access.

This was not hypothetical. The repository's own design-trail evidence (`docs/superpowers/evidence/m1.13-design-trail/m1.13-findings.md`, I9/I11) reproduced it directly: a repo-local slow `clean` filter, `git add -A` SIGKILLED mid-filter, `pgrep -x git` empty afterward, `.git/index.lock` still present. `exec_stage_all`

(`planner.rs`) still runs plain `run_git` — no timeout, no `kill_on_drop` — so a killed server process, an OOM-kill, or a `systemd restart` mid-`StageAll` can leave exactly this behind today.

Answering "not busy" is not sufficient by itself. Verified empirically against real git 2.43 before writing any implementation: with a stale, unheld `index.lock` left on disk, a plain `git add` still fails with its own `fatal: Unable to create '.../index.lock': File exists.` — git's own lockfile creation is `O_CREAT|O_EXCL` and does not check whether anything still holds the file it is refusing to overwrite. A fix that only changes `refuse_if_git_busy`'s answer, without removing the orphan, trades one permanent refusal for another with worse wording.

Decision

1. **Verify liveness via `/proc` before trusting existence.** A new `index_lock_is_open_by_a_live_process` walks `/proc/<pid>/fd`, comparing each open file descriptor's `(device, inode)` against the lock path's — identity, not the path string, because the lock's directory entry can be removed and recreated by an unrelated process while the original holder's fd (and the file object it points to) still exists. Linux-only, matching every other part of this server (landlock, seccomp, the sandbox shim) — there is no non-Linux target to serve.
2. **A confirmed-stale lock is removed, not merely bypassed.** Once no process is found holding it open, `refuse_if_git_busy` unlinks the file before answering "not busy" — the only way the *next* real git command can succeed, per the `O_EXCL` finding above. Removal is safe precisely because liveness was checked first: nothing that could still write through that fd exists to have the file pulled out from under it.

3. **Fails safe in both direction and scope.** Any error `stat`-ing the lock path or reading `/proc` at all makes the check answer "live" (assume busy) rather than risk declaring a real in-progress write stale. A single process's `/proc/<pid>/fd` being unreadable (it exited between the listing and the read, or belongs to another user) is treated as *no evidence* for that one process, not as "not holding it" — the scan keeps checking the rest before concluding stale.
4. **This is a general preflight fix, not a per-operation one.** Unlike ADR 0058's `run_git_hooked` (which bounds `CommitOnHead`/`AmendCommit` specifically and found, by measurement, that those two never leave `index.lock` behind), this fix lives in the shared `refuse_if_git_busy` that every mutating operation calls before executing. It closes the gap for `StageAll`'s still-unbounded `run_git`, for a genuine external terminal git that crashes, and for any future timeout/kill path on `MergeBranch`/`CheckoutBranch`/`RebaseOntoBase` — without needing each of those to separately reason about lock cleanup.
5. **The narrower rebase/merge/checkout gap stays open, deliberately.** `refuse_if_git_busy` still checks only `index.lock`, not `rebase-merge`/`rebase-apply`/`MERGE_HEAD`/`CHERRY_PICK_HEAD`. A killed rebase leaves the repository detached-HEAD with none of those markers caught by this preflight at all — a different, already-documented defect (same evidence file, lines 39-41) that this decision does not touch. See "Consequences" below.

Alternatives considered

- **Age/mtime threshold instead of a liveness probe.**

Rejected: a legitimate slow operation (a large repack, a genuinely slow hook) could exceed any fixed threshold, and the m1.13 evidence file explicitly warns against "a blanket delete any stale lock, which would race a legitimate external git." A liveness probe answers the actual question instead of guessing from elapsed time.

- **Report "not busy" without removing the file.** Rejected by measurement — see Context above: git's own `0_EXCL` lockfile creation still fails against an orphan left on disk, so this would not fix the user-facing defect at all, only change which permanent error the user sees.
- **Checkpoint-side cleanup:** have the process that *kills* its own spawned git (`run_git_hooked`'s `kill_on_drop`) record and remove exactly the lock path it may have orphaned, scoped to kills it performed itself. This is the mechanism the m1.13 finding originally prescribed, and it is *safer* in one respect — no race window between checking and removing, because the killer knows it just killed the only possible holder. Not chosen here because it only protects operations that go through a bounded, killed spawn (today, only `CommitOnHead/AmendCommit`, which ADR 0058 already proved don't hit this case) and does nothing for `StageAll` or a genuine external git crash — the cases this defect is actually about. A future bounded `StageAll` could add this as defense in depth; it would not replace the liveness check here.
- **Extending the same liveness check to `rebase-merge/MERGE_HEAD` in this pass.** Deferred, not rejected — see Decision point 5. That gap needs its own recovery story (an `--abort` on the affected operation, not just a file removal — see the m1.13 finding's own three-part prescribed fix), not a rename of this one.

Consequences

- A repository with an orphaned `index.lock` — from a killed `StageAll` filter/hook, an OOM-kill, or a crashed external terminal git — recovers on its own on the next request, instead of being refused forever.
- The check now costs a bounded `/proc` walk on the (rare) path where `index.lock` exists at all; idle repositories — the overwhelming common case — pay nothing extra, since the

existence check still short-circuits first.

- The one race this introduces: between confirming no process holds the lock and unlinking it, a *new* process could theoretically take it. This is the same class of race the function's own doc comment already accepts for the "not busy" case generally ("the external process can take the lock in the moment after this returns") — narrower here, since it requires a new process to appear in the handful of syscalls between the scan and the unlink, not just after the whole function returns.
- `MergeBranch`, `CheckoutBranch`, and `RebaseOntoBase` remain able to leave the repository in a state this preflight does not detect at all (`rebase-merge`, `MERGE_HEAD`) — an open, separately-scoped gap, named here rather than silently left unaddressed.
- Two `coordination_suite.rs` tests that previously simulated "an external git is busy" by writing an unheld `index.lock` directly now hold it open with a real `git add` behind a slow repo-local `clean` filter (the exact mechanism the m1.13 evidence trail measured), since an unheld lock is now — correctly — recognized as stale rather than busy.

Where this is implemented

- `refuse_if_git_busy`, `index_lock_is_open_by_a_live_process` — `crates/git-vista-server/src/coordinator.rs`.
- Tests — `crates/git-vista-server/src/coordinator.rs`:
`an_index_lock_held_by_a_live_process_marks_the_repository_busy`,
`a_stale_index_lock_does_not_refuse_the_repository_forever`.
- Updated fixtures — `crates/git-vista-server/src/planner/coordination_suite.rs`:
`a_repository_busy_with_an_external_git_is_refused`,
`the_busy_check_finds_a_linked_worktrees_own_index_lock`.